



CAHIER DES CHARGES

Système de Transaction — Banque Mobile

Champ	Information
Intitulé du projet	Système de Transaction Bancaire Mobile
Type de banque	Banque Mobile
Utilisateurs cibles	Clients particuliers
Version	1.0
Date	11 avril 2026
Statut	Document de référence — Devoir ICT304

Proposer par :

ATIWA KUETE Elsa Lauraine 23V2352

Superviser par :

Pr. KIMBI Xaveria

Année Académique : 2025-2026

1. Présentation du Projet

1.1 Contexte

Dans le cadre de ce projet, qui porte sur la conception d'un système de transaction pour une banque mobile. L'objectif est de développer une API REST permettant aux clients particuliers de gérer leurs comptes bancaires depuis leur téléphone : créer un compte, consulter la liste des comptes, effectuer des dépôts et des retraits.

1.2 Objectifs du projet

- Permettre la création de comptes bancaires en ligne
- Offrir la consultation de la liste des comptes
- Permettre les opérations de dépôt et de retrait
- Assurer la traçabilité de toutes les transactions
- Garantir la sécurité et l'intégrité des données financières

1.3 Périmètre

Ce cahier des charges couvre uniquement la couche API back-end du système. L'interface mobile (front-end) et l'intégration aux systèmes interbancaires sont hors périmètre de la version 1.0.

2. Acteurs du Système

Les acteurs sont les entités qui interagissent avec le système bancaire.

Acteur	Rôle	Actions autorisées
Client particulier	Titulaire d'un compte bancaire mobile	Créer un compte, déposer, retirer, consulter
Administrateur	Agent de la banque mobile	Gérer tous les comptes, superviser les transactions
Système (API)	Application back-end Node.js	Traiter les requêtes, valider les données, répondre

3. Expression des Besoins

3.1 Besoins Fonctionnels

Les besoins fonctionnels décrivent CE QUE le système doit faire.

BF-01 — Gestion des comptes

- Le client peut créer un compte en fournissant : nom, prénom, email, type de compte
- Chaque compte reçoit un identifiant unique et un numéro de compte généré automatiquement
- Le client peut consulter la liste de tous les comptes
- Le client peut consulter le détail d'un compte spécifique

BF-02 — Opérations financières

- Le client peut effectuer un dépôt d'argent sur son compte
- Le client peut effectuer un retrait d'argent depuis son compte
- Un retrait est refusé si le solde est insuffisant
- Chaque opération met à jour le solde du compte en temps réel

BF-03 — Traçabilité

- Toute transaction (dépôt ou retrait) est enregistrée avec : type, montant, date, solde après opération
- L'historique des transactions est consultable par compte

3.2 Besoins Non Fonctionnels

Les besoins non fonctionnels décrivent COMMENT le système doit fonctionner.

ID	Catégorie	Exigence
BNF-01	Performance	Le système répond en moins de 5 s pour 95% des requêtes
BNF-02	Disponibilité	Le service est disponible 99,9% du temps (24h/24, 7j/7)
BNF-03	Sécurité	Toutes les communications passent par HTTPS (TLS 1.2+)
BNF-04	Intégrité	Aucun solde ne peut être négatif sans autorisation explicite
BNF-05	Validité	Toutes les données saisies par le client sont vérifiées avant traitement
BNF-06	Scalabilité	L'architecture est sans état pour permettre la mise à l'échelle
BNF-07	Maintenabilité	Le code est documenté, modulaire et couvert par des tests
BNF-08	Conformité	Respect du RGPD (Règlement Général sur la Protection des Données) pour le traitement des données personnelles

ID	Catégorie	Exigence
BNF-09	Auditabilité	Toutes les opérations sensibles sont journalisées
BNF-10	Interopérabilité	L'API respecte les standards REST (Representational State Transfer) et retourne du JSON

4. Cas d'Utilisation

Un cas d'utilisation décrit une interaction entre un acteur et le système pour atteindre un objectif précis.

CU-01 — Créer un compte

Champ	Description
Acteur	Client particulier
Objectif	Ouvrir un nouveau compte bancaire mobile
Précondition	L'adresse email ne doit pas déjà exister dans le système
Déclencheur	Le client envoie une requête POST /api/comptes
Flux principal	1. Client envoie : nom, prénom, email, type de compte, solde initial 2. Système valide les données 3. Système génère un ID et un numéro de compte 4. Compte créé avec succès — réponse 201
Flux alternatif	Données manquantes ou invalides → Erreur 400
Flux alternatif	Email déjà utilisé → Erreur 409
Postcondition	Le compte est créé et actif dans le système

CU-02 — Lister les comptes

Champ	Description
Acteur	Client / Administrateur
Objectif	Obtenir la liste de tous les comptes bancaires
Précondition	Aucune
Déclencheur	Requête GET /api/comptes
Flux principal	1. Requête reçue 2. Système récupère tous les comptes 3. Retourne la liste en JSON avec le nombre total
Postcondition	Liste des comptes retournée avec succès

CU-03 — Effectuer un dépôt

Champ	Description
Acteur	Client particulier
Objectif	Ajouter de l'argent sur son compte
Précondition	Le compte existe et est actif
Déclencheur	Requête POST /api/comptes/:id/dépôt
Flux principal	1. Client envoie le montant 2. Système vérifie le compte 3. Solde mis à jour 4. Transaction enregistrée — réponse 200
Flux alternatif	Montant invalide (négatif ou nul) → Erreur 400
Flux alternatif	Compte introuvable → Erreur 404
Postcondition	Le solde du compte est augmenté et la transaction tracée

CU-04 — Effectuer un retrait

Champ	Description
Acteur	Client particulier
Objectif	Retirer de l'argent depuis son compte
Précondition	Le compte existe, est actif et le solde est suffisant
Déclencheur	Requête POST /api/comptes/:id/retrait
Flux principal	1. Client envoie le montant 2. Système vérifie le solde 3. Solde décrémenté 4. Transaction enregistrée — réponse 200
Flux alternatif	Solde insuffisant → Erreur 422
Flux alternatif	Compte introuvable → Erreur 404
Postcondition	Le solde du compte est diminué et la transaction tracée

5. Spécifications de l'API REST

5.1 Informations générales

Propriété	Valeur
Technologie	Node.js + Express.js
Format des données	JSON (application/json)
Base URL	http://localhost:3000/api
Méthodes HTTP	GET, POST
Codes de statut	200, 201, 400, 404, 409, 422, 500

5.2 Liste des endpoints

Méthode	Route	Description	Code succès
POST	/api/comptes	Créer un nouveau compte	201
GET	/api/comptes	Lister tous les comptes	200
GET	/api/comptes/:id	Détail d'un compte	200
POST	/api/comptes/:id/depot	Effectuer un dépôt	200
POST	/api/comptes/:id/retrait	Effectuer un retrait	200
GET	/api/comptes/:id/transactions	Historique des transactions	200

5.3 Modèle de données — Compte

Champ	Type	Description	Obligatoire
id	UUID	Identifiant unique auto-généré	Auto
numeroCompte	String	Numéro de compte auto-généré	Auto
nom	String	Nom du titulaire	Oui
prenom	String	Prénom du titulaire	Oui
email	String	Email unique du client	Oui
typeCompte	Enum	courant ou epargne (défaut: courant)	Non
solde	Number	Solde courant en francs CFA / euros	Auto
statut	Enum	actif, suspendu ou cloture	Auto
dateCreation	DateTime	Date et heure de création (ISO 8601)	Auto

5.4 Codes d'erreur

Code HTTP	Signification	Exemple de cas
400	Données invalides	Champ obligatoire manquant, montant négatif
404	Ressource introuvable	ID de compte inexistant
409	Conflit	Email déjà utilisé par un autre compte
422	Non traitable	Solde insuffisant pour le retrait demandé
500	Erreur serveur	Erreur inattendue côté serveur

6. Règles Métier

1. Un client ne peut avoir qu'un seul compte par adresse email.
2. Le solde d'un compte ne peut jamais être négatif.
3. Les montants des dépôts et retraits doivent être strictement supérieurs à zéro.
4. Toute transaction est définitive et irréversible une fois confirmée.
5. Le numéro de compte est généré par le système et ne peut pas être modifié.
6. L'historique des transactions doit être conservé pendant 10 ans minimum.

7. Architecture Technique

7.1 Stack technologique

Composant	Technologie	Rôle
Runtime	Node.js v22	Environnement d'exécution JavaScript côté serveur
Framework	Express.js 4	Gestion des routes et des requêtes HTTP
Identifiants	UUID v4	Génération d'identifiants uniques
Stockage	Mémoire (v1.0)	Données stockées en RAM pour la démonstration
Base de données	PostgreSQL	Stockage persistant prévu pour la version 2.0

7.2 Structure du projet

- index.js — Point d'entrée, configuration Express et toutes les routes
- package.json — Dépendances et configuration du projet Node.js
- node_modules/ — Librairies installées (express, uuid...)